

CURSO DE JAVA

CAPÍTULO 1

SINTAXIS

Desde principiante hasta nivel práctico

Nivel	Desde principiante hasta nivel práctico
Enfoque	Desarrollo de Java
Tipo de material	Capítulo técnico y educativo
Objetivo	Comprender, practicar y aplicar la sintaxis de Java

Material diseñado para estudio autónomo, práctica progresiva y construcción de bases sólidas para los siguientes capítulos.

Objetivo del capítulo

La sintaxis es el conjunto de reglas que indican cómo debe escribirse un programa para que el compilador de Java pueda leerlo, analizarlo y transformarlo en código ejecutable. En este capítulo aprenderás esas reglas desde cero: estructura de un programa, clases, método main, instrucciones, bloques, llaves, punto y coma, comentarios, identificadores, palabras reservadas, convenciones de nombres y lectura de errores de sintaxis.

La meta no es memorizar símbolos aislados. La meta es desarrollar la capacidad de leer código, escribirlo correctamente, detectar errores y construir programas pequeños con una estructura profesional.

Al finalizar este capítulo serás capaz de:

- Explicar qué significa sintaxis en Java y por qué el lenguaje necesita reglas formales.
- Reconocer la estructura mínima de un programa Java.
- Identificar clases, métodos, bloques, instrucciones y expresiones.
- Usar correctamente llaves {}, paréntesis (), corchetes [] y punto y coma ;.
- Distinguir comentarios de código ejecutable.
- Crear identificadores válidos y aplicar convenciones profesionales de nombres.
- Reconocer palabras reservadas y evitar usarlas como identificadores.
- Leer mensajes básicos de errores de compilación relacionados con sintaxis.
- Escribir programas pequeños y ordenados siguiendo una estructura consistente.
- Relacionar la sintaxis con sistemas reales como máquinas, sensores, inventarios y automatización.

1. Introducción a la sintaxis

Imagina que quieres darle instrucciones a un robot industrial. No basta con saber qué quieres que haga: necesitas expresar las órdenes en una forma que el controlador pueda interpretar. Un lenguaje de programación funciona de manera parecida. Java necesita que las instrucciones respeten reglas precisas.

Analogía: la sintaxis de Java se parece a la gramática de un idioma. En español, una frase mal construida puede resultar difícil de entender. En Java, una estructura incorrecta puede impedir que el compilador construya el programa.

La sintaxis no existe para complicar el aprendizaje. Existe para eliminar ambigüedades. El compilador necesita saber exactamente dónde comienza una clase, dónde termina un método, qué elementos forman una instrucción y qué símbolos tienen un significado especial.

En proyectos reales, la sintaxis aparece en todas partes: aplicaciones empresariales, servicios web, sistemas de producción, software de automatización, programas de control, videojuegos, aplicaciones Android y herramientas internas.

Si no existieran reglas sintácticas, el compilador tendría que adivinar qué quiso decir el programador. Eso produciría interpretaciones ambiguas y programas mucho más difíciles de verificar.

2. La estructura mínima de un programa Java

Un programa Java puede crecer hasta tener miles de clases y millones de líneas, pero sus componentes básicos siguen patrones reconocibles. Para empezar, estudia esta estructura mínima:

```
public class Main {  
  
    public static void main(String[] args) {  
        System.out.println("Hola, Java");  
    }  
  
}
```

Aquí aparecen varias piezas fundamentales: una clase llamada Main, un método llamado main, un bloque delimitado por llaves y una instrucción que muestra texto en consola.

2.1 ¿Qué es una clase?

Una clase es una estructura que permite definir un tipo de objeto y organizar código relacionado. En este capítulo basta con verla como un contenedor estructurado donde Java puede encontrar atributos y métodos.

La palabra **public** indica que la clase tiene acceso público. **class** le dice a Java que estamos declarando una clase. **Main** es el identificador elegido para esa clase.

```
public class Main {  
    // contenido de la clase  
}
```

2.2 ¿Qué es un método?

Un método es un bloque de código que representa una acción o comportamiento. En Java, los métodos tienen una declaración que describe aspectos como su acceso, tipo de retorno, nombre y parámetros.

2.3 El método main

El método main es el punto de entrada tradicional de una aplicación Java ejecutada directamente. Su forma más común es:

```
public static void main(String[] args) {\n    // instrucciones\n}
```

No necesitas memorizar cada palabra todavía. En capítulos posteriores se estudiarán public, static, void, parámetros, tipos y arreglos con mayor profundidad. Aquí lo importante es reconocer la estructura.

3. Símbolos fundamentales de la sintaxis

Java utiliza símbolos para delimitar y organizar el código. Aprender a identificarlos reduce muchos errores iniciales.

Símbolo	Nombre	Función básica	Ejemplo
{ }	Llaves	Delimitan bloques de código.	class Main { }
()	Paréntesis	Agrupan parámetros o expresiones.	main(args)
[]	Corchetes	Se usan, entre otras cosas, en arreglos.	String[] args
;	Punto y coma	Marca el final de muchas instrucciones.	x = 10;
" "	Comillas dobles	Delimitan literales de texto.	"Hola"
//	Comentario de línea	Permite escribir una anotación ignorada por el compilador.	// comentario
/* */	Comentario de bloque	Permite comentar varias líneas.	/* texto */

3.1 Llaves { }

Las llaves delimitan bloques. Un bloque es un conjunto de instrucciones que pertenece a una estructura concreta. Las llaves deben abrir y cerrar correctamente.

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Inicio");  
    }  
}
```

Observa que hay un bloque para la clase y otro bloque interno para main. La indentación ayuda a visualizar esta relación, aunque los espacios no son lo que determina el bloque: las llaves sí.

3.2 Punto y coma ;

Muchas instrucciones de Java terminan con punto y coma. Es parecido a colocar un punto al final de una oración, aunque no todos los elementos del lenguaje lo usan.

```
System.out.println("Producción iniciada");  
System.out.println("Sensor activo");
```

Un error típico es olvidar el punto y coma:

```
System.out.println("Sensor activo")  
// Falta ;
```

El compilador normalmente señalará una ubicación cercana al problema. Esa ubicación no siempre coincide exactamente con el carácter que causó el error; por eso conviene revisar también la línea anterior.

4. Instrucciones, expresiones y bloques

Una **instrucción** (statement) es una unidad de trabajo que el programa puede ejecutar. Una **expresión** es una combinación de elementos que produce un valor. Un **bloque** agrupa instrucciones mediante llaves.

Elemento	Idea sencilla	Ejemplo
Instrucción	Orden completa para realizar una acción.	System.out.println("OK");
Expresión	Construcción que puede producir un valor.	5 + 3
Bloque	Conjunto de instrucciones delimitado por llaves.	{ ... }
Declaración	Introduce algo en el programa.	int piezas = 10;

La distinción será cada vez más importante. Por ahora, úsala para describir qué papel cumple cada parte del código.

5. Comentarios

Un comentario es texto escrito dentro del código para explicar, documentar o dejar notas. El compilador no lo ejecuta como una instrucción.

5.1 Comentario de una línea

```
// Este comentario explica la siguiente instrucción  
System.out.println("Máquina activa");
```

5.2 Comentario de varias líneas

```

/*
    Este bloque puede contener
    varias líneas de documentación.
*/
System.out.println("Sistema listo");

```

Los comentarios son útiles cuando aportan contexto. No deben utilizarse para explicar obviedades repetidamente. El objetivo profesional es que el código sea claro por sí mismo y que los comentarios expliquen decisiones, reglas o información que no resulta evidente.

6. Identificadores y nombres

Un identificador es el nombre que el programador asigna a elementos como clases, variables y métodos. Java impone reglas para que esos nombres puedan reconocerse correctamente.

Reglas básicas:

- Puede comenzar con una letra, guion bajo (_) o signo de dólar (\$), aunque en código profesional normalmente se prefieren letras.
- Después del primer carácter pueden aparecer letras y dígitos.
- No puede contener espacios.
- No puede ser una palabra reservada de Java.
- Java distingue mayúsculas y minúsculas: Sensor y sensor son identificadores diferentes.

Ejemplos:

Nombre	¿Válido?	Comentario
temperatura	Sí	Nombre claro.
temperaturaMotor	Sí	Convención camelCase.
2sensores	No	No puede comenzar con dígito.
temperatura motor	No	Contiene espacio.
class	No	Es palabra reservada.
Sensor	Sí	Pero puede representar una clase según la convención.

6.1 Convenciones de nombres

Elemento	Convención habitual	Ejemplo
Clase	PascalCase	ControladorMotor
Método	camelCase	mostrarEstado()
Variable	camelCase	temperaturaActual
Constante	MAYÚSCULAS_CON_GUIONES	MAX_TEMPERATURA

7. Palabras reservadas

Las palabras reservadas tienen un significado especial dentro del lenguaje. Por eso no puedes utilizarlas libremente como nombres.

```
public class Main {
    public static void main(String[] args) {
        int piezas = 100;
        System.out.println(piezas);
    }
}
```

Ejemplos de palabras reservadas que encontrarás a lo largo del curso: class, public, private, static, void, int, double, boolean, if, else, switch, while, for, return, new, extends, implements, interface, abstract y final.

No necesitas aprenderlas todas de memoria en este capítulo. Aprenderás su significado conforme aparezcan.

8. Espacios, indentación y legibilidad

Java permite muchos espacios y saltos de línea sin cambiar el significado del código. Sin embargo, la legibilidad sí cambia.

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Código legible");
    }
}
```

La indentación muestra visualmente qué instrucciones están dentro de qué bloques. Una práctica común es usar cuatro espacios por nivel de indentación. Lo importante en un proyecto es mantener un estilo consistente.

Código difícil de leer:

```
public class Main{public static void main(String[]args){System.out.println("Hola");}}
```

Aunque una forma compacta puede ser válida sintácticamente, dificulta el mantenimiento. En software profesional, legibilidad y consistencia son parte de la calidad.

9. Ejemplos progresivos

Nivel 1 — Programa mínimo

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Sistema iniciado");
    }
}
```

Línea por línea: la primera línea declara la clase; la segunda declara main; la tercera ejecuta una llamada para imprimir texto; las llaves delimitan los bloques.

Resultado esperado: aparece **Sistema iniciado** en la consola.

Nivel 2 — Varias instrucciones

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Máquina: M-01");
        System.out.println("Estado: ACTIVA");
        System.out.println("Producción: 250 piezas");
    }
}
```

```
}
```

Aquí se combinan estructura, instrucciones y literales de texto. Cada llamada termina con ;.

Nivel 3 — Caso práctico industrial

```
public class MonitoreoMaquina {
    public static void main(String[] args) {
        System.out.println("=== MONITOREO ===");
        System.out.println("Máquina: M-01");
        System.out.println("Sensor: Temperatura");
        System.out.println("Estado: NORMAL");
        System.out.println("Producción: 850 piezas");
    }
}
```

Este programa todavía no lee sensores reales. Es una simulación para practicar la estructura. En capítulos posteriores podrás reemplazar valores fijos por variables, entradas y lógica.

Nivel 4 — Estructura de una aplicación pequeña

```
public class SistemaProduccion {

    public static void mostrarEncabezado() {
        System.out.println("=== SISTEMA DE PRODUCCIÓN ===");
    }

    public static void mostrarMaquina() {
        System.out.println("Máquina: M-01");
        System.out.println("Estado: ACTIVA");
    }

    public static void main(String[] args) {
        mostrarEncabezado();
        mostrarMaquina();
    }
}
```

Aquí aparece una idea profesional importante: separar responsabilidades. Cada método contiene una tarea concreta. Todavía no se estudian métodos en profundidad, pero observar esta estructura prepara el terreno para los siguientes capítulos.

10. Errores de sintaxis y lectura del compilador

Un error de sintaxis ocurre cuando el código no respeta las reglas del lenguaje. El compilador intenta analizar el programa y muestra mensajes para indicar dónde encontró un problema.

Errores frecuentes:

Error	Causa	Cómo corregirlo
';' expected	Falta punto y coma o existe una estructura incompleta.	Revisar la instrucción y la línea anterior.
'}' expected	Falta cerrar un bloque.	Contar y revisar las llaves.
class expected	La estructura alrededor de una declaración es incorrecta.	Revisar llaves, declaraciones y posición del código.
cannot find symbol	Nombre mal escrito o elemento no disponible en ese contexto.	Revisar mayúsculas, ortografía y declaración.
reached end of file while parsing	La estructura quedó abierta.	Buscar llaves, paréntesis o bloques sin cerrar.

Una estrategia profesional es leer primero el primer error. Un solo error puede provocar muchos mensajes secundarios. Corrige de arriba hacia abajo y vuelve a compilar.

11. Errores de confusión comunes

Concepto	Se confunde con	Diferencia
Clase	Método	La clase organiza una estructura; el método representa una acción o comportamiento.
{ }	()	Las llaves delimitan bloques; los paréntesis agrupan parámetros o expresiones.
;	}	El punto y coma termina muchas instrucciones; la llave cierra bloques.
Comentario	Código	El comentario es ignorado por el compilador como instrucción ejecutable.
Identificador	Palabra reservada	El identificador es un nombre elegido por el programador; la reservada tiene significado propio.
Mayúsculas/minúsculas	Ortografía irrelevante	Java es sensible a mayúsculas y minúsculas.

Ejemplo de confusión:

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hola");  
    }  
}
```

Las dos llaves finales no son dos instrucciones. Una cierra main y la otra cierra la clase. Visualizar la anidación ayuda a entenderlo.

12. Buenas prácticas

Usa nombres descriptivos

Un nombre como temperaturaMotor comunica más información que t. Esto reduce errores y facilita el mantenimiento.

Mantén una indentación consistente

La estructura visual permite identificar rápidamente qué pertenece a cada bloque.

Una responsabilidad por método

Separar tareas facilita probar, modificar y reutilizar el código.

Evita comentarios innecesarios

Comenta decisiones o reglas que no sean evidentes; no repitas literalmente lo que ya expresa el código.

Corrige primero el primer error

Los errores posteriores pueden ser consecuencias del primero.

Mantén un formato uniforme

Un estilo consistente facilita trabajar en equipo y revisar cambios.

No memorices sin comprender

La sintaxis se vuelve natural con lectura y práctica repetida.

13. Conectando conocimientos

La sintaxis es la base sobre la que se construyen los siguientes capítulos. Primero aprendes a escribir una estructura válida; después esa estructura recibirá datos, tomará decisiones, repetirá acciones, organizará información y modelará objetos.

Etapas	Qué se construye
Sintaxis	Estructura válida del programa.
Variables y tipos	Datos almacenados y manipulados.
Operadores	Cálculos y comparaciones.
Condicionales	Decisiones.
Bucles	Repetición controlada.
Métodos	Organización y reutilización de comportamiento.
Arreglos	Colecciones de datos del mismo tipo.
POO	Modelado de entidades y relaciones.

14. Ejercicios prácticos

No mires una solución completa antes de intentarlo. El objetivo es desarrollar la capacidad de construir y corregir el código.

Ejercicio 1 — Comprensión básica

Objetivo: Reconocer la estructura mínima.

Enunciado:

Crea una clase llamada Main con un método main que muestre en consola:

Sistema iniciado.

No uses variables ni condiciones.

Requisitos:

- La clase debe llamarse Main.
- main debe tener su estructura convencional.
- La instrucción de salida termina con ;.

Pistas:

- Revisa que cada { tenga su } correspondiente.
- Comprueba el punto y coma.

Ejercicio 2 — Aplicación

Objetivo: Practicar varias instrucciones.

Enunciado:

Crea un programa que muestre:

=== MÁQUINA ===
Código: M-02
Estado: ACTIVA
Producción: 500 piezas

Requisitos:

- Usa una clase.
- Usa main.
- Cada dato debe aparecer en una instrucción de salida.

Pistas:

- Escribe primero la estructura de clase y main.
- Después agrega una línea de salida a la vez.

Ejercicio 3 — Integración

Objetivo: Integrar sintaxis, nombres y comentarios.

Enunciado:

Crea una clase SistemaInventario. Dentro de main muestra el nombre del sistema, agrega un comentario que explique el objetivo del programa y muestra tres productos.

Requisitos:

- Nombre de clase en PascalCase.
- Comentario de una línea o bloque.
- Código correctamente indentado.

Pistas:

- El comentario no necesita ejecutarse.
- Revisa mayúsculas y minúsculas del nombre de la clase.

Ejercicio 4 — Caso práctico

Objetivo: Aplicar sintaxis a un sistema Java.

Enunciado:

Diseña la estructura inicial de un programa llamado MonitoreoRobot. Debe mostrar un encabezado, el identificador del robot, el sensor supervisado y un estado inicial.

Requisitos:

- Clase MonitoreoRobot.
- Método main.
- Al menos cuatro salidas.
- Formato legible.

Pistas:

- Piensa en cada salida como una instrucción independiente.
- No necesitas sensores reales todavía: es una simulación.

Ejercicio 5 — Reto

Objetivo: Detectar y corregir errores de sintaxis.

Enunciado:

Analiza el siguiente programa y corrige todos los errores sin cambiar su objetivo:

```
public class Control {  
    public static void main(String[] args) {  
        System.out.println("Inicio")  
        System.out.println("Sensor activo");  
        System.out.println("Producción: 100")  
    }  
}
```

Requisitos:

- No cambies los textos.
- Corrige únicamente los problemas sintácticos.
- Conserva la estructura de clase y main.

Pistas:

- Busca instrucciones que no terminan correctamente.
- Comprueba las llaves.
- Compila nuevamente después de cada corrección.

15. Mini proyecto del capítulo

Nombre del proyecto: Sistema de inicio de una máquina industrial

Contexto: una planta necesita un programa Java que muestre información inicial de una máquina antes de comenzar un proceso. En este capítulo no controlarás hardware real; construirás la interfaz inicial de consola.

Objetivo: crear una estructura Java correcta y legible que sirva como base para ampliar el sistema en capítulos posteriores.

Requisitos funcionales:

- Mostrar un encabezado del sistema.
- Mostrar el código de la máquina.
- Mostrar el estado inicial.
- Mostrar el sensor que será supervisado.

- Mostrar una línea indicando que el sistema está listo.
- Organizar el código mediante una clase y el método main.

Requisitos técnicos:

- Java.
- Una clase principal.
- Método main.
- Instrucciones de salida.
- Indentación consistente.
- Comentarios solo cuando aporten información útil.

Paso 1

Crea el archivo/clase principal y establece la estructura mínima.

Paso 2

Agrega el encabezado y los datos de identificación de la máquina.

Paso 3

Agrega sensor y estado inicial.

Paso final

Revisa llaves, paréntesis, punto y coma, nombres y formato. Ejecuta el programa y verifica que el resultado sea claro.

Pistas: construye el programa de arriba hacia abajo. No intentes escribir todo de una vez. Después de agregar cada bloque, compila y corrige los errores antes de continuar.

16. Comprueba tus conocimientos

1. ¿Qué es la sintaxis de un lenguaje de programación?
2. ¿Por qué Java necesita reglas sintácticas?
3. ¿Qué función cumplen las llaves {}?
4. ¿Qué función cumple normalmente el punto y coma ;?
5. ¿Qué es una clase?
6. ¿Qué es un método?
7. ¿Cuál es la función del método main en una aplicación Java tradicional?
8. ¿Qué diferencia hay entre un comentario y una instrucción ejecutable?
9. ¿Qué es un identificador?
10. ¿Por qué temperaturaMotor es un nombre válido y 2motor no?

11. ¿Java distingue entre Sensor y sensor?
12. ¿Qué es una palabra reservada?
13. ¿Por qué la indentación es importante aunque Java use llaves para delimitar bloques?
14. Si aparece un error cerca de una llave, ¿por qué conviene revisar también las líneas anteriores?
15. Analiza: `System.out.println("OK")` ¿qué problema sintáctico presenta?
16. ¿Qué estrategia conviene seguir cuando el compilador muestra muchos errores?

17. Respuestas y explicaciones

1. La sintaxis es el conjunto de reglas que define cómo debe escribirse el código para que el lenguaje pueda analizarlo.
2. Porque el compilador necesita una estructura inequívoca para interpretar el programa.
3. Delimitan bloques de código, como el cuerpo de una clase o un método.
4. Termina muchas instrucciones de Java.
5. Una estructura que organiza elementos relacionados y puede definir un tipo de objeto.
6. Un bloque declarado para representar una acción o comportamiento.
7. Es el punto de entrada habitual de una aplicación Java ejecutada directamente.
8. El comentario sirve como documentación o anotación y no representa una instrucción ejecutable.
9. Es el nombre asignado a elementos del programa, como clases, métodos o variables.
10. Un identificador no puede comenzar con un dígito.
11. Sí. Java es sensible a mayúsculas y minúsculas.
12. Es una palabra cuyo significado está reservado por el lenguaje y que no puede utilizarse libremente como identificador.
13. Porque mejora la legibilidad y hace visible la jerarquía de bloques.
14. Porque el error real puede estar en una línea anterior; un carácter faltante puede desordenar el análisis posterior.
15. Falta el punto y coma al final de la instrucción.
16. Corregir primero el primer error reportado, volver a compilar y continuar de arriba hacia abajo.

18. Resumen del capítulo

Concepto	Para qué sirve	Cuándo utilizarlo	Error común
Sintaxis	Define la estructura válida del código.	Siempre.	Escribir estructuras incompletas.
Clase	Organiza código y define una estructura.	En programas Java.	Olvidar llaves.
Método main	Punto de entrada habitual.	Al ejecutar una aplicación directamente.	Escribir incorrectamente su estructura.
Llaves	Delimitan bloques.	Clases, métodos y otras estructuras.	No cerrar un bloque.

Concepto	Para qué sirve	Cuándo utilizarlo	Error común
Punto y coma	Termina muchas instrucciones.	En instrucciones que lo requieren.	Olvidarlo.
Comentarios	Documentan y explican.	Cuando aportan contexto.	Usarlos para describir obviedades.
Identificadores	Nombran elementos.	Clases, métodos, variables, etc.	Usar palabras reservadas o nombres inválidos.
Indentación	Mejora legibilidad.	Siempre en proyectos mantenibles.	Formato inconsistente.

Ideas clave que debes recordar

- La sintaxis es el conjunto de reglas que permite que Java interprete el código.
- Una clase y un método están delimitados por bloques de llaves.
- El método main es el punto de entrada habitual de una aplicación Java ejecutada directamente.
- Muchas instrucciones terminan con punto y coma.
- Java distingue mayúsculas y minúsculas.
- Los identificadores deben respetar reglas del lenguaje.
- Las palabras reservadas tienen significado especial.
- Los comentarios sirven para documentar, no para ejecutar instrucciones.
- La indentación no sustituye a las llaves, pero mejora enormemente la lectura.
- Ante errores de compilación, conviene corregir primero el primer error y volver a compilar.

19. Mapa de aprendizaje

La progresión de este curso puede entenderse como una construcción por capas:

Etapas	Resultado
Conocimiento previo	Comprender qué es un programa y reconocer código fuente.
Nuevo concepto: Sintaxis	Escribir estructuras que Java pueda analizar.
Aplicación básica	Crear una clase, main e instrucciones simples.
Aplicación práctica	Construir una salida organizada para un sistema realista.
Integración en proyectos	Combinar sintaxis con datos, decisiones y repetición.
Uso profesional	Mantener código legible, consistente y fácil de ampliar.

Este capítulo prepara directamente el camino para variables, tipos de datos, entrada y salida, operadores, decisiones, ciclos, métodos, arreglos y programación orientada a objetos. La sintaxis no desaparece al avanzar: se convierte en la base invisible de todo lo demás.

20. Lista de verificación final

- Puedo explicar qué significa sintaxis.
- Puedo identificar una clase.

- Puedo identificar el método main.
- Puedo reconocer un bloque delimitado por llaves.
- Sé cuándo aparece normalmente el punto y coma.
- Puedo escribir comentarios de una y varias líneas.
- Sé distinguir un identificador de una palabra reservada.
- Puedo aplicar nombres legibles y convenciones básicas.
- Puedo detectar errores sencillos de llaves y punto y coma.
- Puedo crear un programa Java pequeño desde cero.
- Puedo leer un mensaje básico del compilador y buscar la causa.
- Puedo explicar cómo este conocimiento prepara los siguientes temas.